

Activity 5

Public Key Infrastructure

6733172621 Patthadon Phengpinij

Collaborators. Claude (for L^AT_EX styling and grammar checking)

Overview

In this activity, you will learn the fundamentals of Public Key Infrastructure. We will need the following tools:

- A. OpenSSL. On Linux and Mac OS X, the OpenSSL is installed by default. For Windows, you may download it from <https://wiki.openssl.org/index.php/Binaries>.
- B. Python with PyOpenSSL and pem to do our exercise. If your python does not come with PyOpenSSL and pem, you may install it with pip

```
1 $ pip install pyopenssl
2 $ pip install pem
```

- C. You also need ca-certificates.crt from your OS (e.g. /etc/cacerts/ca-certificates.crt in Linux) or take it from the course web.

Exercise

Issuing the following command.

```
1 $ openssl s_client -connect www.chula.ac.th:443
2 $
```

Once connected, you may try

```
1 GET / HTTP/1.0
2 [Enter twice]
```

(Note that the server may return HTTP 404. This is completely normal since we did not send a request for a valid resource.)

Repeat the same step again, now with

```
1 $ openssl s_client -connect www.chula.ac.th:443 -CAfile ca-certificates.crt
2 $
```

This command basically connects to port 443 (HTTPS) with the TLS/SSL. This is like a standard telnet command, but with openssl performing the encryption for you.

Problem 1. From the two given openssl commands, what is the difference? Note: If your operating system does not show any error in the first command, try

```
1 $ openssl s_client -connect www.chula.ac.th:443 -CApath empty_dir
2 $
```

If the results are still the same, your system is not reliable. You may ignore this exercise. (Modern versions of Mac OS X will always read CA from keychains. There is no intuitive way to turn it off.)

Solution

Answer: the certificate chain is the same, but only the command with `-CAfile ca-certificates.crt` can verify it.

(On Ubuntu the first command reads the system store by default and also says ok, so I used `-CApath empty_dir` for it.)

```
# -CApath empty_dir (no trusted root)
depth=2 C = GB, O = Sectigo Limited, CN = Sectigo Public Server Authentication Root R46
verify error:num=20:unable to get local issuer certificate
Verify return code: 20 (unable to get local issuer certificate)

# -CAfile ca-certificates.crt
depth=2 C = GB, O = Sectigo Limited, CN = Sectigo Public Server Authentication Root R46
depth=1 C = GB, O = Sectigo Limited, CN = Sectigo Public Server Authentication CA DV R36
depth=0 CN = *.chula.ac.th
Verify return code: 0 (ok)
```

Both connections still work (TLS is set up and the server answers the GET). The difference is only whether the client can **trust** the server's identity.

Problem 2. What does the error (verify error) in the first command mean? Please explain.

Solution

Answer: unable to get local issuer certificate means OpenSSL followed the chain up to Sectigo Public Server Authentication Root R46, but could not find a **trusted root** for it in its local store (the store is empty).

So the chain cannot end at a CA that we trust, and the server could be anyone (e.g. a man-in-the-middle). The connection is encrypted, but the server is not authenticated.

Problem 3. Copy the server certificate (beginning with -----BEGIN CERTIFICATE----- and ending with -----END CERTIFICATE-----) and store it as `www_chula_ac_th.cert`. Use the command

```
1 $ openssl x509 -in www_chula_ac_th.cert -text
2 $
```

to show a text representation of the certificate content. Briefly explain what is stored in an X.509 certificate (i.e. data in each field).

Solution

Answer: an X.509 certificate binds a **public key** to an **identity**, signed by a CA. Main fields from `openssl x509 -in www_chula_ac_th.cert -text`:

Field	Value (www.chula.ac.th) / meaning
Version, Serial Number	v3; a unique number given by the CA (used in revocation)
Signature Algorithm	sha256WithRSAEncryption: how the CA signed it
Issuer	Sectigo Public Server Authentication CA DV R36: the CA that signed it
Validity	5 Jan 2026 – 5 Feb 2027
Subject	CN = *.chula.ac.th: the owner
Subject Public Key Info	RSA 2048-bit public key of the server
Key Usage / Extended Key Usage	Digital Signature, Key Encipherment / TLS server (and client) authentication
Basic Constraints	CA:FALSE: cannot sign other certificates
Subject Alternative Name	*.chula.ac.th, chula.ac.th: hostnames it is valid for
Authority Information Access	where to get the issuer cert (CA Issuers) and check revocation (OCSP)
Certificate Policies	DV policy (2.23.140.1.2.1): only the domain was validated
CT SCTs	proof that it was logged in Certificate Transparency logs
Signature Value	the CA's signature over all the fields above

Problem 4. From the information in exercise 3, is there an intermediate certificate? If yes, what purpose does it serve?

Hint: Look for an issuer and download the intermediate certificate. You may use the command:

```
1 $ openssl x509 -inform der -in intermediate.cert -text
2 $
```

to show the details of the intermediate certificate. (Note that the `-inform der` is for reading the DER file. The default file format for x509 is the PEM file.)

Solution

Answer: yes. The issuer of the server certificate is Sectigo Public Server Authentication CA DV R36, which is not a root. I downloaded it from the CA Issuers URL in the certificate:

```
$ curl -o intermediate.cert http://crt.sectigo.com/SectigoPublicServerAuthenticationCADVR36.crt
$ openssl x509 -inform der -in intermediate.cert -text
Issuer: C = GB, O = Sectigo Limited, CN = Sectigo Public Server Authentication Root R46
Subject: C = GB, O = Sectigo Limited, CN = Sectigo Public Server Authentication CA DV R36
X509v3 Basic Constraints: critical
CA:TRUE, pathlen:0
X509v3 Key Usage: critical
Digital Signature, Certificate Sign, CRL Sign
```

Purpose: the intermediate CA signs the end-entity (server) certificates *on behalf of* the root. The root private key can then be kept **offline**. If the intermediate is compromised, only it has to be revoked, not the root that is built into every OS/browser. The chain is: *.chula.ac.th ← CA DV R36 ← Root R46 (in `ca-certificates.crt`).

Problem 5. Is there an intermediate CA, i.e. is there more than one organization involved in the certification? Say why you think so.

Solution

Answer: there is an intermediate CA, but it belongs to the same organization as the root. Only one organization does the certification.

The intermediate (CA DV R36) and the root (Root R46) both have O = Sectigo Limited. The intermediate is just Sectigo's own sub-CA. The server also sends a cross-certificate of Root R46 signed by USERTrust RSA Certification Authority, but USERTrust is also owned by Sectigo. Chulalongkorn University is only the subject, not a CA.

Problem 6. What is the role of ca-certificates.crt?

Solution

Answer: it is the **trust store**, the list of root CA certificates that we trust in advance (shipped with the OS).

A chain is accepted only if it ends at one of these roots. This is why `-CAfile ca-certificates.crt` gives Verify return code: 0 (ok) and an empty store gives error 20.

Problem 7. Explore the ca-certificates.crt. How many certificates are in there? Give the command/method you have used to count.

Solution

Answer: 121 certificates (ca-certificates.crt from Ubuntu 22.04).

Every certificate starts with exactly one BEGIN CERTIFICATE line, so I count those lines:

```
$ grep -c "BEGIN CERTIFICATE" ca-certificates.crt
121
$
```

Problem 8. Extract a root certificate from ca-certificates.crt. Use the openssl command to explore the details. Do you see any Issuer information? Please compare it to the details of www.chula.ac.th's certificate and the details of the intermediate certificate.

Solution

Answer: yes, it has an Issuer, but Issuer = Subject (the root is **self-signed**). I split the bundle into one file per certificate and picked Sectigo Public Server Authentication Root R46 (the root of the www.chula.ac.th chain):

```
$ awk '/BEGIN CERT/{n++; n{print > ("roots/" n ".pem")}' ca-certificates.crt
$ for f in roots/*.pem; do openssl x509 -in $f -noout -subject | grep -q "Root R46$" && cp $f root_sectigo_r46.pem; done
$ openssl x509 -in root_sectigo_r46.pem -text
```

	Server (*.chula.ac.th)	Intermediate (CA DV R36)	Root (Root R46)
Issuer	CA DV R36	Root R46	Root R46 (itself)
Basic Constraints	CA:FALSE	CA:TRUE, pathlen:0	CA:TRUE
Key Usage	Digital Sig., Key Encipherment	Cert Sign, CRL Sign	Cert Sign, CRL Sign
Public key	RSA 2048	RSA 3072	RSA 4096
Validity	about 1 year (to 2027)	15 years (to 2036)	25 years (to 2046)

Nobody signs the root for us to check it. It is trusted only because it is already in the trust store. Going up the chain, the certificates also get bigger keys and longer lifetimes.

Problem 9. If the intermediate certificate is not in a PEM format (text readable), use the following command to convert a DER file (.cert .cer .der) to PEM file:

```
1 $ openssl x509 -inform der -in certificate.cer -out certificate.pem
2 $
```

(You need the pem file for exercise 10.)

Solution

The downloaded intermediate is DER (binary), so I converted it to PEM:

```
$ openssl x509 -inform der -in intermediate.cer -out intermediate.pem
$ head -1 intermediate.pem
-----BEGIN CERTIFICATE-----
```

Problem 10. From the given python code^[1], implement the certificate validation.

```
1 from OpenSSL import crypto
2 import pem
3
4 def verify():
5     with open("./target.cert", "r") as cert_file:
6         cert = cert_file.read()
7
8     with open("./intermediate.cert", "r") as int_cert_file:
9         int_cert = int_cert_file.read()
10
11     pems = pem.parse_file("./ca-certificates.cert");
12     trusted_certs = []
13     for mypem in pems:
14         trusted_certs.append(str(mypem));
15
16     trusted_certs.append(int_cert);
17
18     verified = verify_chain_of_trust(cert, trusted_certs)
19
20     if verified:
21         print("Certificate verified")
22
23
24 def verify_chain_of_trust(cert_pem, trusted_cert_pems):
25     certificate = crypto.load_certificate(crypto.FILETYPE_PEM, cert_pem)
26
27     # Create and fill a X509Store with trusted certs
28     store = crypto.X509Store()
29     for trusted_cert_pem in trusted_cert_pems:
30         trusted_cert = crypto.load_certificate(crypto.FILETYPE_PEM, trusted_cert_pem)
31         store.add_cert(trusted_cert)
32
33     # Create a X509StoreContext with the cert and trusted certs
34     # and verify the the chain of trust
35     store_ctx = crypto.X509StoreContext(store, certificate)
36     # Returns None if certificate can be validated
37     result = store_ctx.verify_certificate()
38
39     if result is None:
40         return True
41     else:
42         return False
```

Use your program to verify the certificates of:
www.chula.ac.th, google, classdeedee.cloud.cp.eng.chula.ac.th

Solution

Answer: all four certificates are verified.

I used the given code with two small changes.

- `verify_certificate()` raises an exception when it fails (it does not return a value), so I catch it and print the reason.
- The intermediate file may contain more than one certificate (the whole chain sent by the server), so it is parsed with `pem.parse_file` as well.

`get_chain.sh` saves `<host>.cert` and `<host>_intermediate.cert` from `openssl s_client -showcerts`.

```

1 def verify(target, intermediate, ca_file="./ca-certificates.crt"):
2     with open(target, "r") as cert_file:
3         cert = cert_file.read()
4         trusted_certs = [str(mypem) for mypem in pem.parse_file(ca_file)]
5         if intermediate:
6             trusted_certs += [str(mypem) for mypem in pem.parse_file(intermediate)]
7         return verify_chain_of_trust(cert, trusted_certs)
8
9 def verify_chain_of_trust(cert_pem, trusted_cert_pems):
10    certificate = crypto.load_certificate(crypto.FILETYPE_PEM, cert_pem)
11    store = crypto.X509Store()
12    for trusted_cert_pem in trusted_cert_pems:
13        store.add_cert(crypto.load_certificate(crypto.FILETYPE_PEM, trusted_cert_pem))
14    store_ctx = crypto.X509StoreContext(store, certificate)
15    try:
16        store_ctx.verify_certificate()
17        return True, "OK"
18    except crypto.X509StoreContextError as e:
19        return False, str(e)

```

```

$ for h in www.chula.ac.th google.com classdeedee.cloud.cp.eng.chula.ac.th; do ./get_chain.sh $h; done
$ python verify.py www.chula.ac.th google.com classdeedee.cloud.cp.eng.chula.ac.th self-signed.badssl.com
www.chula.ac.th           Certificate verified
google.com                Certificate verified
classdeedee.cloud.cp.eng.chula.ac.th Certificate verified
self-signed.badssl.com    NOT verified: self-signed certificate
www_chula_ac_th.cert + intermediate.pem (True, 'OK')
www_chula_ac_th.cert (no intermediate) (False, 'unable to get local issuer certificate')
$

```

The last three lines are checks that the program can also *reject*: a self-signed certificate fails, and the chula certificate fails without its intermediate. **Note:** this code checks only the chain of trust, not the hostname. `classdeedee...` is served the `*.cloud.cp.eng.chula.ac.th` certificate, which still matches through the wildcard (`openssl verify -verify_hostname` also says OK).

Problem 11. Nowadays, there are root certificates for class 1 and class 3. What uses would a class 1 signed certificate have that a class 3 doesn't, and vice versa?

Solution

Answer: the class shows how strongly the CA checked the identity of the owner.

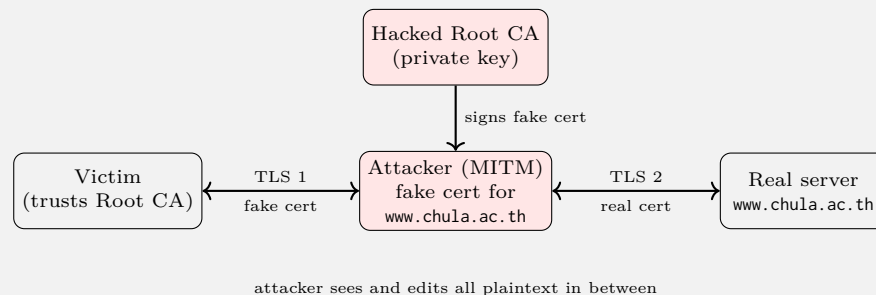
- **Class 1** only checks that you control the e-mail address/domain. It is cheap or free and quick to get, so it is good for personal e-mail signing/encryption (S/MIME), test servers and small personal sites. Class 3 is overkill for these.
- **Class 3** checks the real organization (documents, legal existence). It is used where users must trust *who* is behind the key: banks, e-commerce, government servers, code signing. A Class 1 certificate gives no assurance of identity, so it cannot be used for these.

Problem 12. Assuming that a Root CA in your root store is hacked and under the control of an attacker, and this is not noticed by anyone for months.

- What further attacks can the attacker stage? Draw a possible attack setup.
- In the attack you have described above, can we rely on CRLs or OCSP for protection? Please explain.

Solution

- (a) **Attack:** with the root's private key the attacker can issue a **valid certificate for any domain** (e.g. `www.chula.ac.th`, a bank, Gmail). Every client that trusts this root accepts it. Combined with a man-in-the-middle position (rogue Wi-Fi, ARP/DNS spoofing, BGP hijack, malicious ISP), the attacker can decrypt and modify all HTTPS traffic without any browser warning. They can steal passwords and session cookies or inject malware. They can also sign malware/fake updates (code signing) and fake S/MIME e-mails.



- (b) **No, CRLs and OCSP do not help here.**

- A CRL/OCSP response is produced by the same CA that is compromised. The attacker can sign “good” OCSP responses, or the CA simply does not know about the fake certificates, so they are never listed.
- Nobody has noticed for months, so nothing gets revoked.
- A **root cannot revoke itself**: there is no higher CA to put it on a CRL. The only fix is to remove the root from every trust store (OS/browser update).
- Clients often **soft-fail**: if a MITM blocks the CRL/OCSP request, browsers usually accept the certificate anyway.

Better protections are Certificate Transparency logs (fake certificates become publicly visible) and certificate pinning.